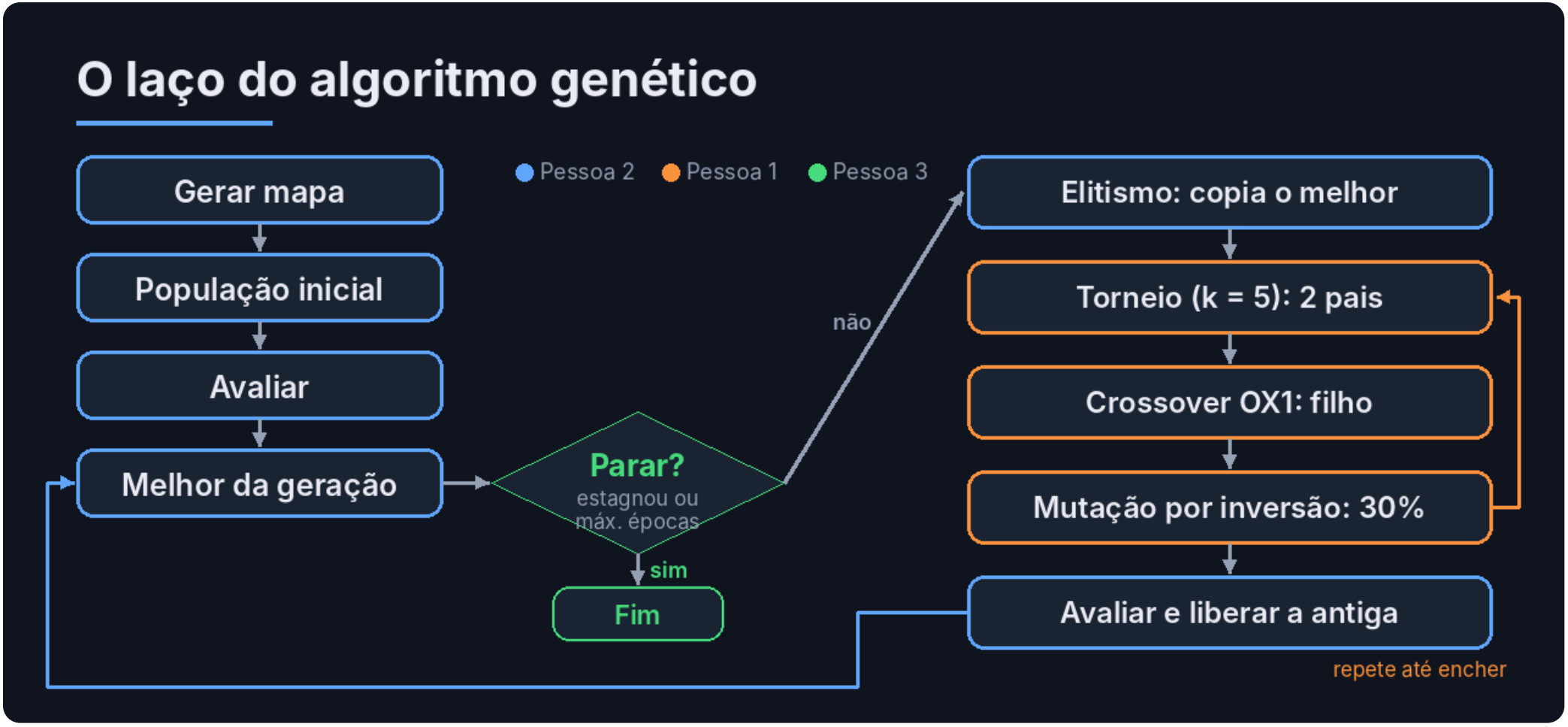
Algoritmo Genético · Caixeiro Viajante

Explicação em 1 minuto

Cada rota é um vetor de índices, ou seja, uma permutação dos pontos. Começamos com rotas aleatórias e medimos cada uma pela distância do ciclo, incluindo a volta ao início; a aptidão é 1 sobre a distância. A cada época, a melhor rota passa intacta (elitismo) e o resto nasce de torneio, cruzamento OX1 e mutação por inversão. Paramos quando a melhor distância estagna. No círculo, cujo ótimo é conhecido, o AG chega ao ótimo exato até 200 pontos.

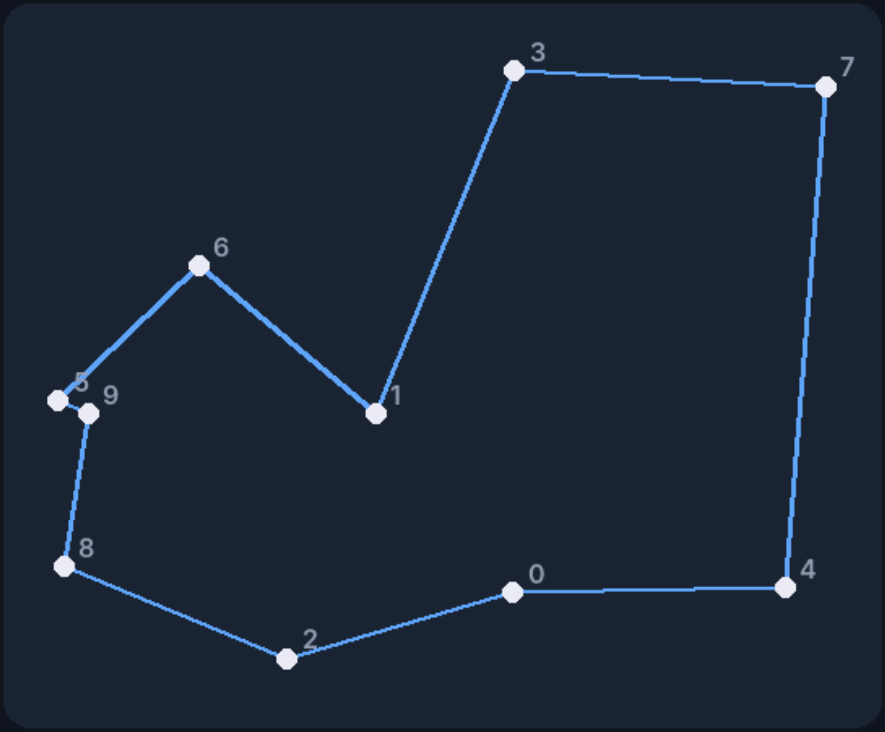
Parâmetros (include/genetic.h)

Representação	vetor de inteiros = permutação de 0..N−1
População	max(150, 2·N), inicial aleatória (Fisher-Yates)
Seleção	torneio, k = 5
Cruzamento	OX1, taxa 100%
Mutação	inversão de trecho, 30% por filho
Elitismo	1 indivíduo
Aptidão	1 / distância do ciclo (com a volta)
Parada	estagnação max(150, 2·N) ou máx. max(1000, 20·N) épocas



- **Antes do laço:** gera o mapa → cria P rotas aleatórias (Fisher-Yates) → avalia (distância e aptidão) → acha a melhor.
- **A cada época:** elitismo copia a melhor → repete *torneio* → *OX1* → *inversão* até encher a população → avalia a nova → libera a antiga.
- **Para** quando a melhor distância fica max(150, 2·N) épocas sem melhorar, ou em max(1000, 20·N) épocas.
- Código: `run_scenario` (main.c) faz o laço; `evolve_population` (genetic.c) cria cada geração.

O problema: caixeiro viajante



- N pontos no plano (x, y)
- Visitar todos, uma vez cada
- Voltar ao ponto de partida (ciclo)
- Minimizar a distância total
- Distância euclidiana: $\sqrt{(dx^2 + dy^2)}$

- N pontos no plano; caminho **cíclico** que passa por todos uma vez e volta ao início; minimizar a distância euclidiana total.
- Por que não força bruta: são (N−1)!/2 rotas. 10 pontos = 181 440; 20 pontos ≈ 60 quatrilhões.
- AG é heurística: acha uma rota boa rápido, sem garantia de ótimo. Por isso o enunciado diz "solucionar **localmente**".

VOCABULÁRIO

Dicionário: biologia → código

Cromossomo / gene	vetor de índices: a rota	`[3, 0, 4, 1, 2]`
Indivíduo	struct Individual	`route + distance + fitness`
População	struct Population	`max(150, 2·N)` indivíduos
Aptidão	fitness	`1 / distância`
Geração / época	uma volta do laço	`evolve_population`
Elitismo	melhor passa intacto	`o melhor nunca piora`

- Indivíduo = uma rota. População = conjunto de rotas. Época = uma volta do laço.
- Aptidão = 1 / distância. Elitismo = a melhor rota passa intacta, então a melhor distância nunca piora.

ITEM: REPRESENTAÇÃO DO GENE (2 PTS, JUNTO COM CRUZAMENTO)

Representação do gene

Pessoa 1 · 2

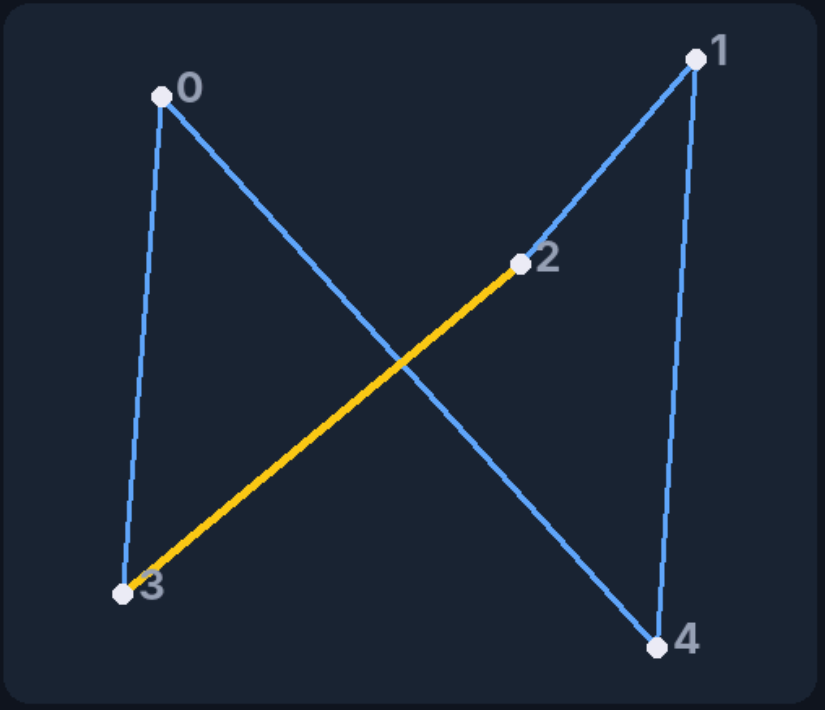
3	0	4	1	2
pos 0	pos 1	pos 2	pos 3	pos 4

Cada número é o índice de um ponto.

A ordem do vetor é a ordem da visita.

3 → 0 → 4 → 1 → 2 → volta ao 3

Sem repetição e sem faltar ninguém: uma permutação.



- Vetor de N inteiros; cada inteiro é o índice de um ponto; a ordem do vetor é a ordem de visita.
- Sempre uma **permutação**: sem repetidos, sem faltar. Os operadores preservam isso, então nunca há indivíduo inválido para consertar.
- Por que vetor: acesso O(1) (o OX1 precisa), inverter trecho é simples, validar é O(N). Codificação binária geraria cidades repetidas.

ITEM: CRUZAMENTO OX1 (PESSOA 1)



- 1) Sorteia um trecho e copia do **pai A** para o filho, mesmas posições. 2) Lê o **pai B** a partir do fim do trecho, dando a volta, e escreve os genes que faltam, pulando os repetidos.
- Filho sempre válido. Preserva um pedaço de rota de A e a ordem relativa de B. Roda em O(N) (tabela `used[gene]`).
- Taxa de cruzamento 100%: o elitismo já protege a melhor rota, então não perdemos nada cruzando sempre.

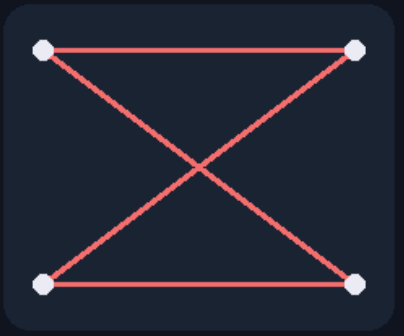


- Sorteia 5 rotas (com reposição); a de **menor distância** vence e vira pai. Repete para o segundo pai.
- Vantagem sobre roleta: não depende da escala da aptidão e um indivíduo muito bom não domina tudo.
- k controla a pressão seletiva: k maior = converge mais rápido, mas perde diversidade.

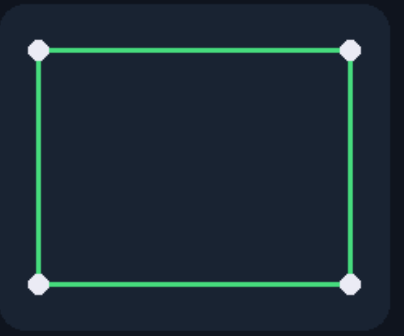
Mutação por inversão (30% por filho)

Pessoa 1

Antes	0	1	2	3	4	5	6	7
Depois	0	1	5	4	3	2	6	7



cruzada



invertida

Sorteia dois índices e inverte o trecho entre eles.

Uma inversão desfaz um cruzamento de arestas, e todo cruzamento indica que a rota pode encurtar.

Só reordena genes que já existem: a rota continua válida.

- Cada filho tem 30% de chance de sofrer **uma** inversão: sorteia dois índices e inverte o trecho. A rota continua válida.
- Inversão troca só 2 arestas e desfaz cruzamentos (cruzar = rota mais longa): mudança pequena e útil.
- Por que 30%: taxa baixa deixa a população igual rápido e trava em ótimo local. Decair a taxa ao longo da execução foi testado e piorou.
- Não destrói boas soluções porque o **elitismo** guarda a melhor intacta.

ITEM: POPULAÇÃO E PARÂMETROS (1 PT COM A PARADA)

Parâmetros (include/genetic.h)

Pessoa 3

População	`max(150, 2·N)`	espaço de busca cresce com N
Torneio	`k = 5`	pressão seletiva moderada
Crossover	`100%`	todo filho mistura dois pais
Mutação	`30% por filho`	mantém diversidade
Elitismo	`1 indivíduo`	melhor nunca piora
Estagnação	`max(150, 2·N) épocas`	parada por convergência
Máx. épocas	`max(1000, 20·N)`	teto de tempo

- População max(150, 2·N): espaço de busca cresce com N; com 100 fixos o AG travava a partir de N = 100. Piso de 150 para mapas pequenos.
- População inicial aleatória (Fisher-Yates) para ter diversidade; sem heurística tipo vizinho mais próximo.

Critério de parada (duplo)

Pessoa 3

Estagnação

max(150, 2·N) épocas seguidas

sem melhorar a melhor distância

Limite

max(1000, 20·N) épocas

teto de segurança

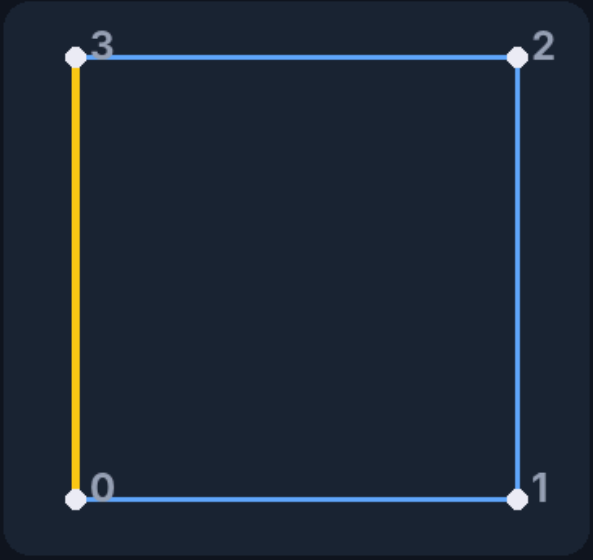
Para no que acontecer primeiro.

Os dois crescem com N: mapa maior precisa de mais paciência.

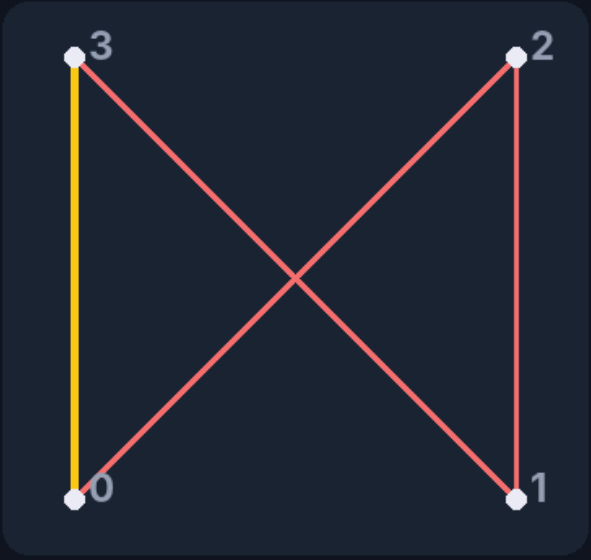
- Duplo: estagnação de max(150, 2·N) épocas sem melhorar **ou** limite de max(1000, 20·N) épocas. O que vier primeiro.
- Crescem com N: mapa maior demora mais entre melhoras. Na prática quase tudo para por estagnação (ex.: círculo N = 20 achou o ótimo na época 32 e parou na 182).

Avaliação: distância do ciclo

Pessoa 2



0,1,2,3 → 4,00



0,2,1,3 → 4,83

cruzou = mais longo

Soma:

```
rota[0] → rota[1]
rota[1] → rota[2]
   …
rota[N−1] → rota[0]
```

Amarelo: a aresta de volta. Sem ela, seria caminho aberto, outro problema.

```
hypot(dx, dy)
```

- Soma dist(rota[i], rota[i+1]) **+ a aresta de volta** rota[N−1] → rota[0]. Sem ela seria caminho aberto, outro problema.
- Quadrado: rota em volta = 4,00; rota cruzada = 2 + 2√2 ≈ 4,83.

Função de aptidão

Pessoa 2

fitness = 1 / distância

	distância	aptidão
Rota A	4,00	0,250
Rota B	4,83	0,207

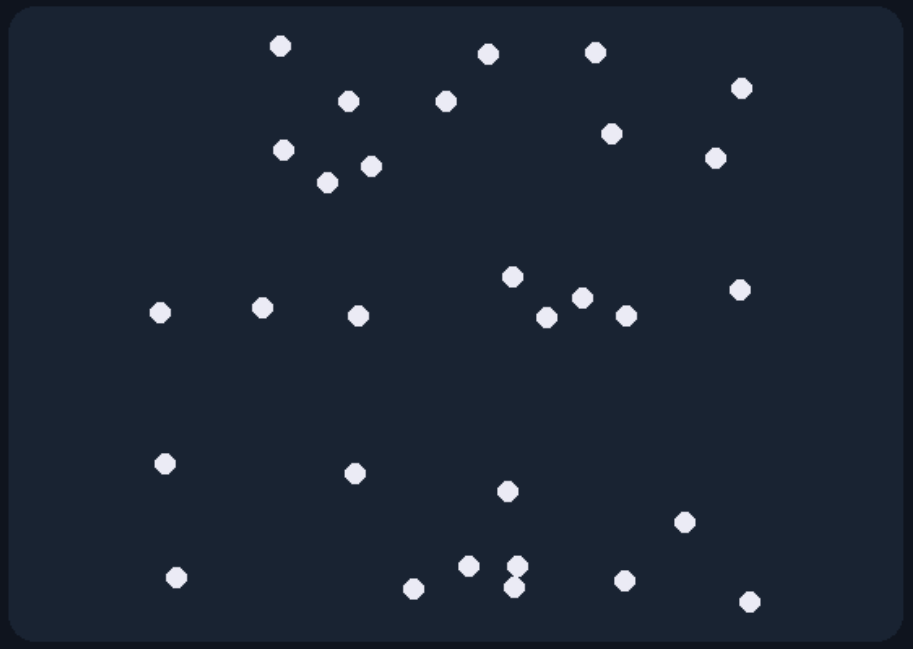
Minimizar a distância = maximizar a aptidão.

Na prática o código compara direto a menor distance: mesma ordem, sem divisão.

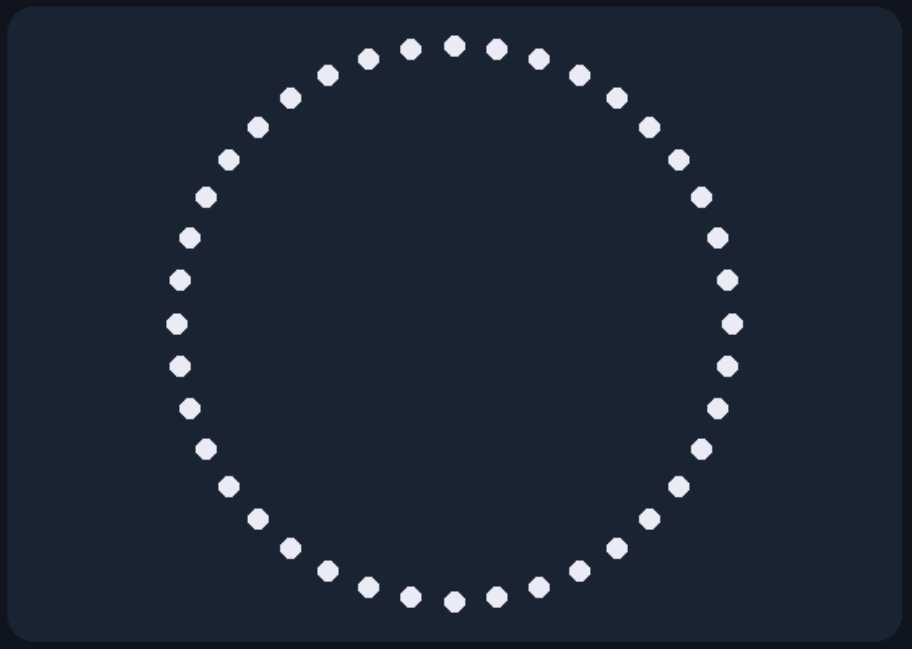
- fitness = 1 / distância. Minimizar distância = maximizar aptidão (1/x é decrescente).
- O código compara direto pela **menor distance** (torneio, melhor, elitismo): mesma ordem, sem divisão.
- Distância ≈ 0 (N < 2 ou pontos iguais) → fitness = 0, para não dividir por zero.

Os dois cenários do enunciado

Pessoa 2



Uniforme: x e y sorteados entre 0 e 100

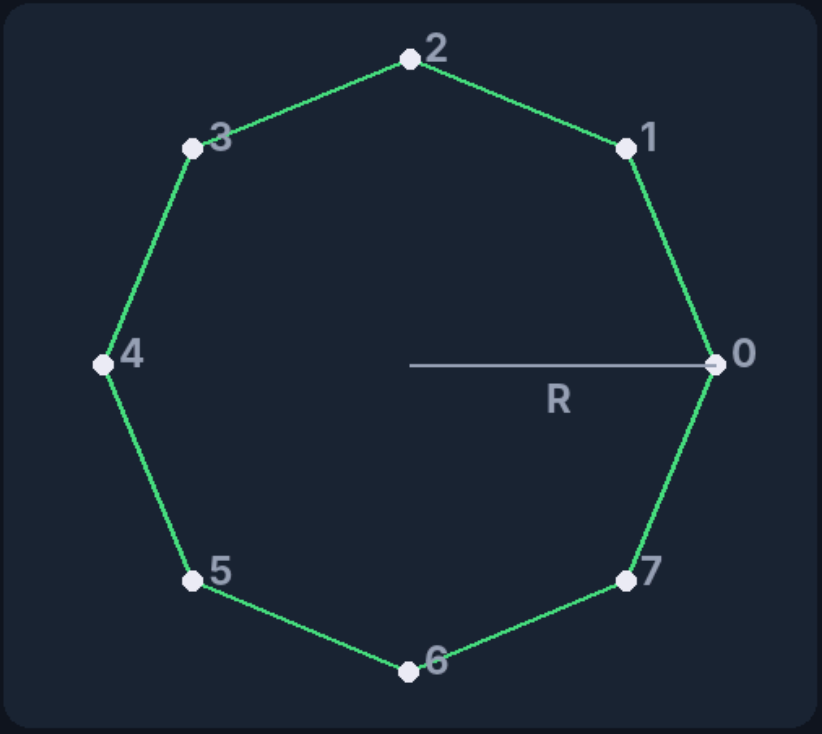


Círculo: ângulo 2π·i/N, raio 100

- Uniforme: x e y sorteados em [0, 100]. Círculo: N pontos igualmente espaçados, ângulo 2π·i/N, raio 100.

Círculo: o benchmark com ótimo conhecido

Pessoa 2



$x = R \cdot \cos(2\pi \cdot i/N)$ $y = R \cdot \sin(2\pi \cdot i/N)$

Melhor rota: dar a volta em ordem

0 → 1 → 2 → … → N−1 → 0

Ótimo = 2 · N · R · sin(π / N)

N = 8, R = 100 → 612,29

Permite medir o gap: (AG − ótimo) / ótimo

- Ótimo conhecido: dar a volta em ordem. Comprimento = 2·N·R·sin(π/N) (perímetro do polígono). N = 8 → 612,29.
- Permite calcular o gap = (melhor do AG − ótimo) / ótimo.

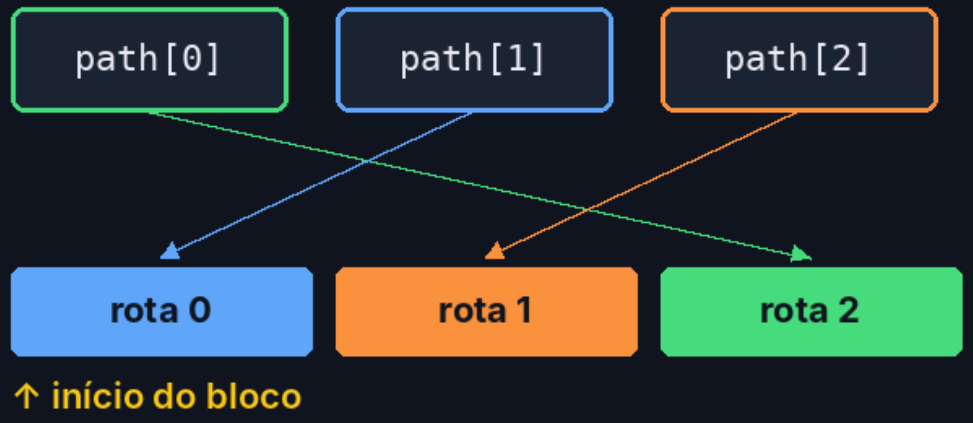


- 3 alocações: struct Population, vetor de P indivíduos e **um bloco** com P × N inteiros. `path[i].route = bloco + i·N`.
- Em vez de P + 2 mallocs: mais rápido, sem vazamento pela metade se um malloc falhar, dados contíguos. Importa porque cria uma população nova a cada época.

Por que guardar route_block?

Pessoa 2

Depois de um qsort (seleção por ranking), os indivíduos trocam de lugar:



Antes

`free(path[0].route)`

agora aponta para o MEIO

→ ASan: bad-free

Depois

`free(pop->route_block)`

O campo route_block nunca muda: a população pode ser reordenada com segurança.

- Se alguém ordenar a população (ex.: qsort para Ranking), path[0] deixa de apontar para o início do bloco; free(path[0].route) quebraria (ASan: bad-free).
- route_block guarda o início para sempre → população pode ser reordenada com segurança.



- Para i de N−1 até 1: sorteia j em [0, i] e troca. Posição i tem i+1 escolhas → N! sequências → cada ordem com chance 1/N!.
- "Trocar pares aleatórios N vezes" tem N^{2N} sequências, que não é múltiplo de N! → viés. Teste: 24 permutações, desvio máx. 2,4%.

Elitismo e cópia profunda

Pessoa 2

Certo: individual_copy

```
memcpy(dst->route, src->route,
       N * sizeof(int));
```

Copia o conteúdo. Cada um tem o seu.

Errado: copiar o ponteiro

```
dst->route = src->route;
```

Dois indivíduos na mesma memória: mexe num, estraga o outro.

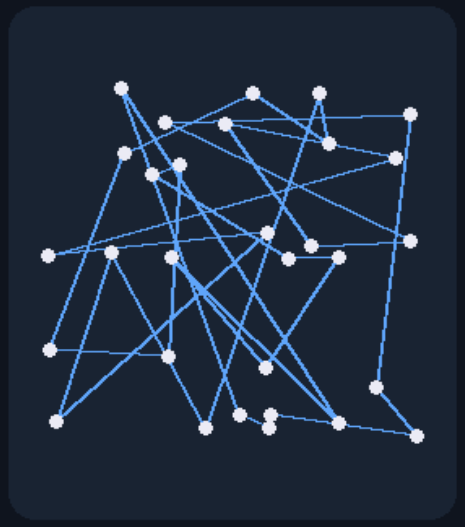
A melhor rota de cada geração é copiada para a próxima.

Resultado: a melhor distância nunca piora de uma época para a outra.

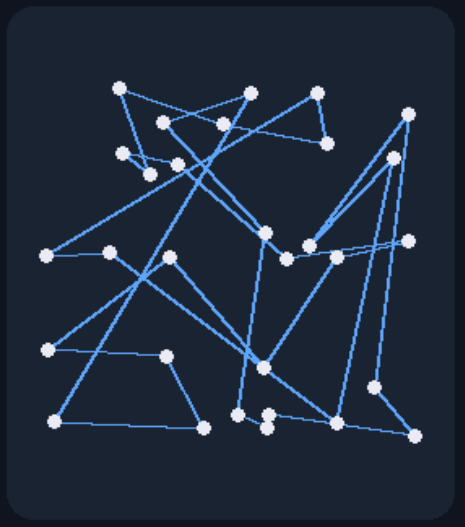
- Melhor rota copiada para a próxima geração com `individual_copy` (memcpy do conteúdo).
- Copiar o ponteiro faria dois indivíduos dividirem memória: mexer num estraga o outro e dá double free.

Uniforme (30 pontos): a melhor rota por época

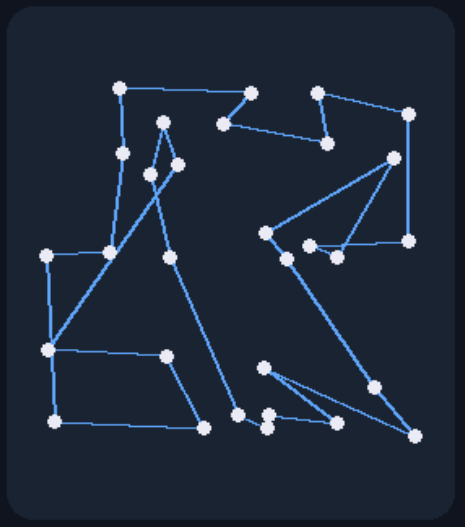
Resultado



época 0
1436,0



época 5
1102,8



época 20
771,2

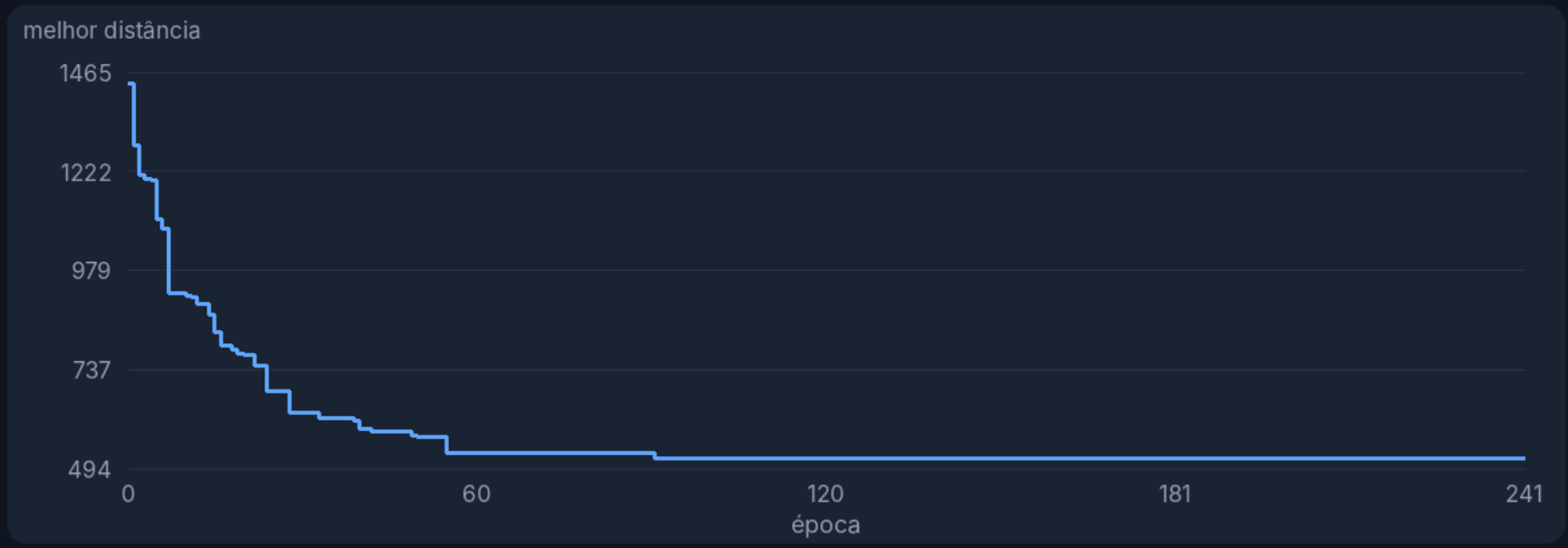


época 91
519,8

- Console: `./genetic_tsp_solver 1 N X [semente]` imprime época, melhor distância, tempo de CPU e rota a cada X épocas (X = 1 → todas). GUI desenha ao vivo.
- Exemplo real: N = 10, semente 42 → 248,65 na época 5 = **ótimo exato** (conferido por força bruta).

Uniforme (30 pontos): melhor distância por época

Resultado



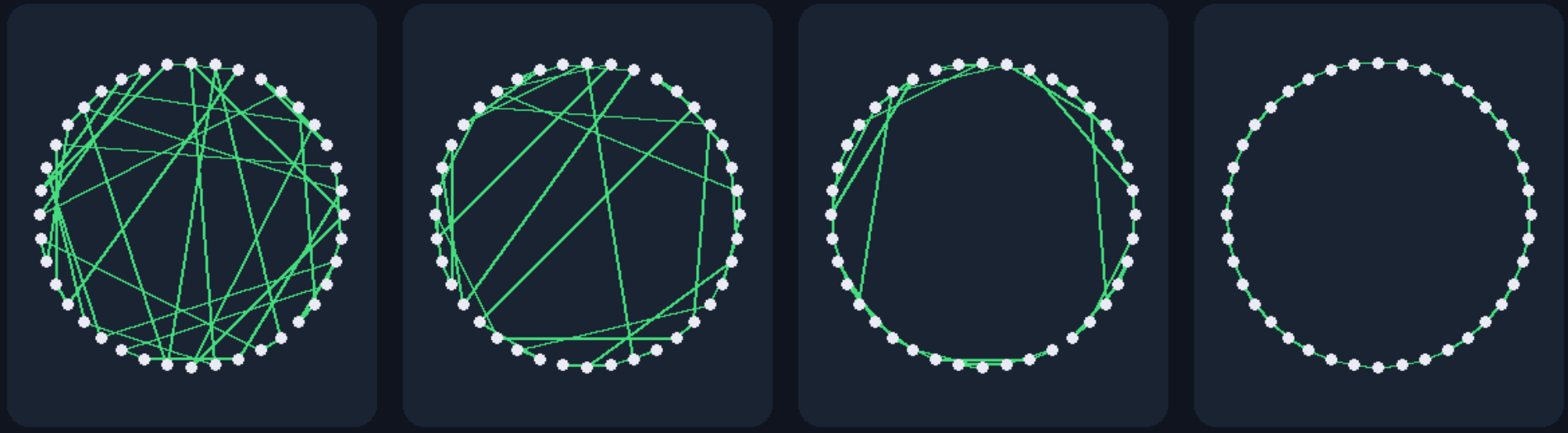
Cai rápido no começo, depois estabiliza: a estagnação encerra a execução.

- Cai rápido no começo e estabiliza; com elitismo nunca sobe; a estagnação encerra.
- Comparado com referência (2-opt, 200 reinícios): N=10 0%; N=30 0 a 9%; N=50 3 a 10%; N=100 6 a 10%. Ótimo local de boa qualidade.

ITEM: DESEMPENHO A CADA ÉPOCA · CÍRCULO (2,5 PTS)

Círculo (40 pontos): a melhor rota por época

Resultado



- Começa como emaranhado; as inversões desfazem os cruzamentos até sobrar a volta em ordem.
- Exemplo real: N = 20, semente 42 → ótimo 625,74 na época 32.



- A curva encosta na linha verde (ótimo teórico): gap 0%.

Resultados medidos

Resultado

Círculo: ótimo exato até 200 pontos

N	ótimo	época	tempo
8	612,29	1	~0 s
50	627,91	205	0,08 s
100	628,22	606	0,51 s
200	628,29	1298	5,2 s
500	gap 0,4%	5806	122 s

Uniforme: comparado com referência

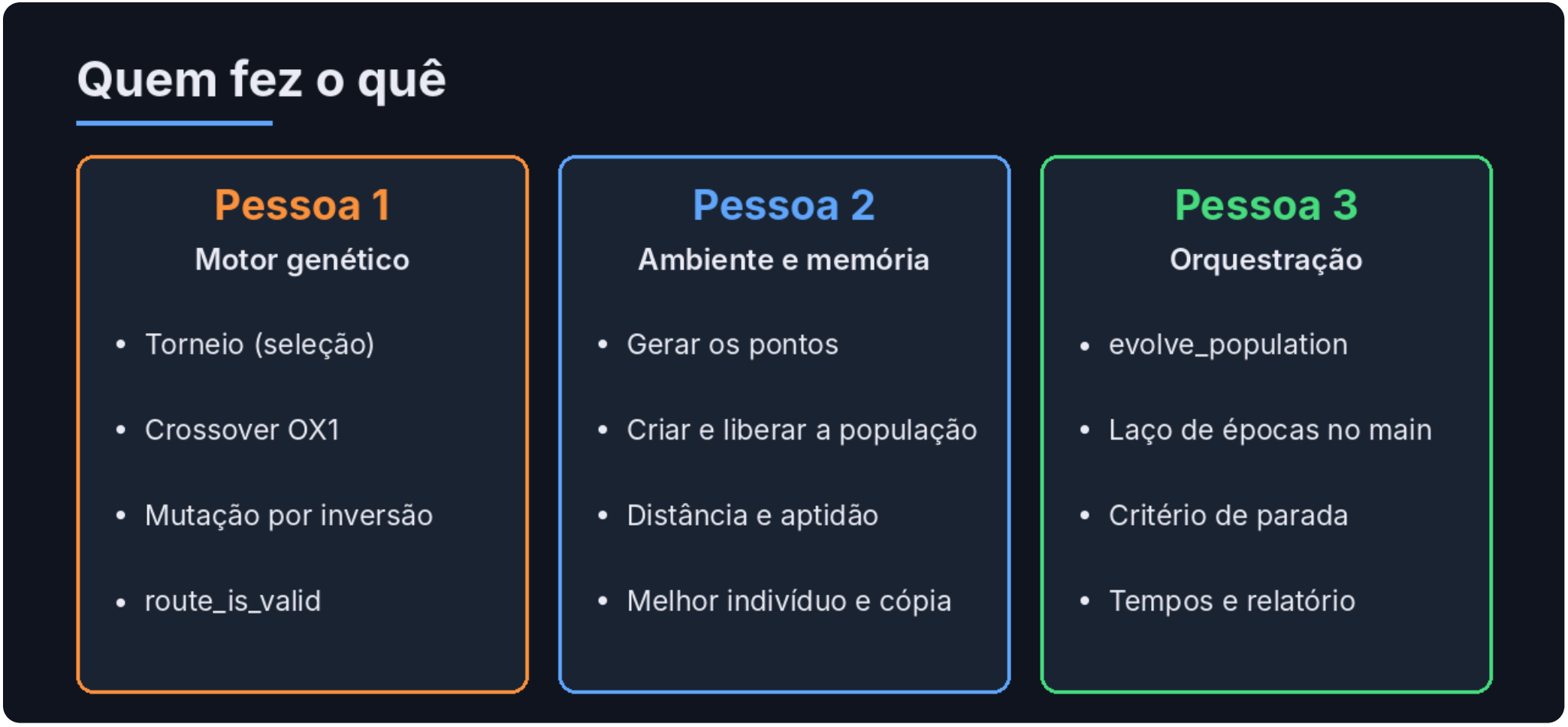
N	distância a mais
10	0% (ótimo exato)
30	0 a 9,2%
50	2,7 a 10,1%
100	5,7 a 9,7%

Ótimo local bom: é o "localmente" do enunciado.

- Círculo: ótimo exato até 200 pontos (200 pontos em 5,2 s). Aleatório: ótimo exato com 10 pontos; poucos % acima com N grande.



- `./genetic_tsp_solver 2 500 500 42`: população 1000, limite 10 000 épocas, estagnação 1000.
- Distância: 19 579 (ép. 500) → 7 292 (2000) → 2 135 (3000) → 769 (4000) → 630,83 (ép. 5806, 122 s). Ótimo 628,31 → **gap 0,40%**.



- Pessoa 1: torneio, OX1, inversão, validação. Pessoa 2: pontos, população/memória, distância/aptidão. Pessoa 3: evolve_population, laço, parada, tempos.

RESPOSTAS RÁPIDAS

Perguntas prováveis

Garante o ótimo?
Não, é heurística ("localmente"). No círculo chega ao ótimo até 200 pontos; no aleatório fica a poucos % da referência.

Por que torneio e não roleta?
Não depende da escala da aptidão; um indivíduo muito bom não domina todas as escolhas.

Por que OX1?
Sempre gera permutação válida; cruzamento comum repetiria cidades.

Por que inversão?
Troca 2 arestas e desfaz cruzamentos; mudança pequena que mantém a rota válida.

Por que 30% de mutação?
Mantém diversidade; taxa baixa trava em ótimo local; o elitismo protege a melhor.

Por que população cresce com N?
Espaço de busca (N−1)!/2 explode; com 100 fixos travava a partir de N = 100.

Por que a aresta de volta?
O caminho é cíclico; sem ela seria caminho aberto.

Por que 1/distância?
Transforma minimização em maximização; na prática compara a menor distância.

Por que bloco único de memória?
3 mallocs em vez de P + 2; sem vazamento parcial; contíguo; população é recriada a cada época.

Por que route_block?
Liberar continua certo mesmo se a população for reordenada (qsort).

Por que memcpy no elitismo?
Copiar ponteiro = memória compartilhada e double free.

Por que Fisher-Yates?
Todas as N! ordens com a mesma chance; trocas aleatórias têm viés.

Complexidade de uma época?
≈ O(P·N): cada filho custa O(N) em OX1, inversão e avaliação.

Por que o círculo é mais fácil?
Cruzamentos são óbvios e cada inversão aproxima da volta em ordem; o aleatório tem muitos ótimos locais.